

Trabalho 5 - Árvores

Identificação do Aluno

- **Nome:** Rafael Lima Sant'Ana
- **Matrícula:** 251035659
- **Disciplina:** FGA0146 - ESTRUTURAS DE DADOS 1
- **Turma:** T04
- **Professor:** MSc. Filipe Emídio Tôrres
- **Instituição:** Universidade de Brasília (UnB) - Faculdade do Gama

Parte 1

1. Escreva a expressão linear dessa árvore.

15(8(4(2, 6), 11(10, 13)), 22(18, 27))

2. Identifiquem:

1. **Raiz:** 15.
2. **Filhos à esquerda e à direita:**
 - **À esquerda:** 8 (de 15), 4 (de 8), 2 (de 4), 10 (de 11), 18 (de 22).
 - **À direita:** 22 (de 15), 11 (de 8), 6 (de 4), 13 (de 11), 27 (de 22).
3. **Folhas:** 2, 6, 10, 13, 18, 27.
4. **Nós internos:** 15, 8, 22, 4, 11.

3. Percorram a árvore em:

1. **Pré-ordem (Raiz, Esquerda, Direita):** 15, 8, 4, 2, 6, 11, 10, 13, 22, 18, 27.
2. **Em-ordem (Esquerda, Raiz, Direita):** 2, 4, 6, 8, 10, 11, 13, 15, 18, 22, 27.
3. **Pós-ordem (Esquerda, Direita, Raiz):** 2, 6, 4, 10, 13, 11, 8, 18, 27, 22, 15.

4. Escrevam o código em C para criar os nós manualmente.

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct No {
    int valor;
    struct No *esq;
    struct No *dir;
} No;
```

```
No* criarNo(int valor) {
    No* novoNo = (No*)malloc(sizeof(No));
    novoNo->valor = valor;
    novoNo->esq = NULL;
    novoNo->dir = NULL;
    return novoNo;
}
```

```

int main() {
    No* raiz = criarNo(15);

    raiz->esq = criarNo(8);
    raiz->dir = criarNo(22);

    raiz->esq->esq = criarNo(4);
    raiz->esq->dir = criarNo(11);

    raiz->dir->esq = criarNo(18);
    raiz->dir->dir = criarNo(27);

    raiz->esq->esq->esq = criarNo(2);
    raiz->esq->esq->dir = criarNo(6);

    raiz->esq->dir->esq = criarNo(10);
    raiz->esq->dir->dir = criarNo(13);

    return 0;
}

```

5. Analisem como a estrutura muda se a inserção for feita em outra ordem.

A estrutura da árvore é altamente dependente da ordem de inserção dos elementos. Se esses mesmos valores fossem inseridos de forma já ordenada (ex: 2, 4, 6, 8...), a árvore não seria balanceada e se tornaria uma árvore "degenerada" (parecendo uma lista encadeada, pendendo apenas para o lado direito). A ordem original garante uma distribuição mais espalhada e balanceada das subárvores.

6. Responda as questões abaixo:

- Qual é a raiz? 15.
- Quais são as folhas? 2, 6, 10, 13, 18, 27.
- Qual é o filho esquerdo de 8? 4.
- Qual é o filho direito de 11? 13.
- Qual é a subárvore enraizada em 8? É a subárvore que tem 8 como raiz, ramificando para os nós esquerdos (4, que liga a 2 e 6) e nós direitos (11, que liga a 10 e 13).
- Qual é o percurso em ordem? 2, 4, 6, 8, 10, 11, 13, 15, 18, 22, 27.

Parte 2

A árvore base para os exercícios 1, 2 e 3 tem raiz 40.

Exercício 1: Análise estrutural

1. Qual é a raiz? 40.
2. Quais são as folhas? 5, 15, 30, 55, 70.
3. Quem é o pai de 55? 50.
4. Qual é a subárvore enraizada em 20? É a subárvore com raiz 20, contendo à esquerda o nó 10 (que tem filhos 5 e 15) e à direita o nó folha 30.
5. Quantos nós internos existem? 5 nós internos (40, 20, 60, 10, 50).

Exercício 2: Percursos

Na mesma árvore do exercício 1, determine:

1. Pré-ordem: 40, 20, 10, 5, 15, 30, 60, 50, 55, 70.
2. Em-ordem: 5, 10, 15, 20, 30, 40, 50, 55, 60, 70.
3. Pós-ordem: 5, 15, 10, 30, 20, 55, 50, 70, 60, 40.

Exercício 3: Identificação de nós

Na árvore do exercício 1:

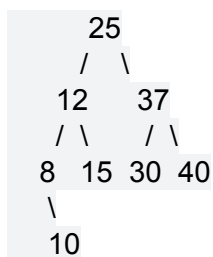
1. Quais são os filhos de 10? 5 e 15.
2. Quais são os irmãos de 50? 70.
3. 30 é nó folha ou nó interno? Nó folha (não possui filhos).
4. 60 tem quantos filhos? Dois filhos (50 e 70).

Exercício 4: Construção da árvore

Construa manualmente uma árvore binária a partir da seguinte estrutura:

raiz: 25, filho esquerdo: 12, filho direito: 37, filho esquerdo de 12: 8, filho direito de 12: 15, filho esquerdo de 37: 30, filho direito de 37: 40, filho direito de 8: 10.

Representação visual da árvore construída:



Exercício 5: Resposta conceitual

Explique por que duas árvores binárias com os mesmos valores podem ser diferentes.

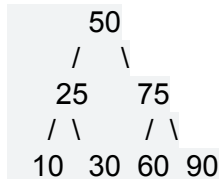
Porque a estrutura de uma árvore depende não apenas dos valores, mas da ordem em que os nós são inseridos e de como a relação de pai e filho é definida. Uma mesma sequência de

números pode gerar uma árvore perfeitamente balanceada ou uma árvore degenerada em linha reta, dependendo inteiramente da cronologia das inserções.

Exercício 6: Desenho a partir de percurso

Uma árvore tem o percurso em pré-ordem: 50 25 10 30 75 60 90. Desenhe uma árvore possível que gere esse percurso.

Assumindo que seja uma Árvore Binária de Busca (BST), a estrutura correta seria:



Exercício 7: Conceitos de memória

Em uma árvore binária implementada em C, responda:

1. **O que acontece com um ponteiro de filho inexistente?** Ele é inicializado para apontar para NULL, indicando o fim daquele ramo.
2. **Por que usamos ponteiros?** Porque as árvores são estruturas de dados dinâmicas. O uso de ponteiros permite alocar memória sob demanda (conforme os nós crescem) em posições não espalhadas na memória, ligando-os através de endereços de memória em vez de usar arrays estáticos de tamanho fixo.
3. **O que precisa ser feito ao final do uso da árvore?** É obrigatório liberar a memória de todos os nós para evitar vazamentos de memória (memory leak). Isso é geralmente feito através da função free() percorrendo a árvore num percurso pós-ordem (apagando os filhos antes do pai).

Exercício 8: Verdadeiro ou falso

1. **Todo nó interno tem exatamente dois filhos.** -> Falso. Um nó interno pode ter apenas um filho.
2. **Uma árvore binária pode ter nós com apenas um filho.** -> Verdadeiro.
3. **A raiz não possui pai.** -> Verdadeiro.
4. **Em percurso em ordem, visita-se primeiro a subárvore direita.** -> Falso. Visita-se primeiro a subárvore esquerda.

Exercício 9: Comparação de estruturas

Considere:

Árvore A: raiz 10, filho esquerdo 5, filho direito 20

Árvore B: raiz 10, filho esquerdo 20, filho direito 5

As árvores são iguais?

Não, elas são estruturalmente diferentes. Embora tenham exatamente a mesma raiz (10) e os mesmos valores de filhos, em árvores binárias a distinção entre "filho esquerdo" e "filho direito" é importante. Na Árvore A, o 5 está à esquerda; na Árvore B, está à direita.

Exercício 10: Desafio de implementação

Escreva a definição de um nó de árvore binária em C e explique o papel de cada campo.

```
typedef struct No {  
    int valor;  
    struct No *esq;  
    struct No *dir;  
} No;
```

Explicação:

- **int valor:** (Campo de Informação/Dado) Armazena a chave ou valor útil que o nó está segurando.
- **struct No *esq:** (Ponteiro Esquerdo) Guarda o endereço de memória do filho à esquerda do nó atual. Se não houver, recebe NULL.
- **struct No *dir:** (Ponteiro Direito) Guarda o endereço de memória do filho à direita do nó atual. Se não houver, recebe NULL.